Modular Continual Network: Growing Capacity for Near-Zero Catastrophic Forgetting

Magnus Tiiso Makgasane

Abstract

We propose the **Modular Continual Network (MCN)**, a novel architecture for task-incremental continual learning that achieves near-zero catastrophic forgetting in the evaluated task-incremental settings by growing dedicated modular capacity per task rather than competing over a fixed weight budget. MCN freezes a shared base encoder after the first task to preserve general low-level representations, and equips each subsequent task with a lightweight CNN adapter module, a learned attention router, and a task-specific output head. On Split-CIFAR-10 (5 tasks), MCN achieves **92.7% average accuracy** with only **0.1% forgetting**, outperforming Elastic Weight Consolidation (60.8 / 39.2% forgetting), PackNet (77.0 / 9.3%), and HAT (59.4 / 43.7%). On Split-CIFAR-100 (20 tasks), MCN scales to **75.1% average accuracy** with **1.5% forgetting**, surpassing all baselines. On Permuted MNIST (5 tasks), MCN reaches 95.9% with 1.2% forgetting. Ablation studies confirm that task-specific modules contribute +17.5% accuracy over the base-only variant, while the attention router adds a further +1.5% through dynamic per-sample feature blending.

1 Introduction

The ability to learn a sequence of tasks without forgetting previously acquired knowledge—*continual learning*—remains a fundamental challenge in deep learning. Neural networks trained sequentially suffer from *catastrophic forgetting* [1]: as gradient descent optimises weights for a new task, it overwrites representations critical to previous tasks.

Three main families of approaches address this problem:

Regularization-based methods (e.g. EWC [2]) add penalty terms that resist changes to important weights. They require no architectural changes but accumulate conflicting constraints as the task count grows, ultimately limiting plasticity.

Parameter isolation methods (e.g. PackNet [3], HAT [5]) allocate disjoint parameter subsets per task via pruning or learned masks. Forgetting is low by construction, yet performance degrades once the fixed capacity is exhausted.

Dynamic architecture methods (e.g. Progressive Neural Networks [4]) grow the network per task. They can greatly reduce forgetting and avoid fixed-capacity limits, but often grow quadratically or require complex lateral connections.

We identify a cleaner path: *fix a general-purpose base encoder after the first task, and grow only the task-specific components.* In the task-incremental setting considered here, this prevents later training stages from directly overwriting the shared representation and gives each task dedicated capacity without competing with others.

Contributions.

- **MCN architecture**: frozen shared encoder + per-task CNN adapters + attention-based router + per-task classification heads.
- **Near-zero forgetting** (0.1%) on Split-CIFAR-10 in task-incremental evaluation at 92.7% average accuracy, outperforming four baselines.
- **Scalability**: 75.1% accuracy with 1.5% forgetting on the 20-task Split-CIFAR-100 benchmark.
- **Ablation study** isolating each component's contribution.
- **Honest evaluation**, including benchmarks where simpler methods are competitive.

2 Related Work

EWC [2] estimates diagonal Fisher Information after each task and penalises deviation from the task-optimal weights. The constraints accumulate across tasks, eventually preventing adaptation.

PackNet [3] prunes and hard-freezes weights per task with binary masks. With a 50% prune fraction, a 5-task sequence exhausts capacity by task 4.

HAT [5] learns per-task attention masks via cumulative gating—a softer version of PackNet's hard masking. In our experiments HAT still exhibits 43.7% forgetting on Split-CIFAR-10, indicating that attention-based masking within fixed capacity is insufficient for visually diverse tasks.

Progressive Neural Networks [4] add one column per task with lateral connections; model size grows as $O(T^2)$.

DEN [6] selectively expands capacity but relies on multiple training phases and heuristic expansion criteria.

MCN differs through *principled separation*: one frozen encoder (trained once, shared forever) plus clean per-task modules (trained independently), avoiding both compounding constraints and capacity exhaustion.

3 Method

3.1 Problem Setting

We consider task-incremental continual learning. A model trains sequentially on T tasks; task t provides $\mathcal{D}_t = \{(x_i, y_i)\}$ with labels $y_i \in \{0, \ldots, C_t - 1\}$. Task identity is known at train and test time. No replay buffer is used.

3.2 MCN Architecture

MCN comprises four components (Figure 1): a shared base encoder, per-task adapter modules, per-task attention routers, and per-task classification heads.

3.2.1 Shared Base Encoder

A standard 3-block CNN with batch normalisation produces a 512-dimensional feature vector. We partition it into two sub-networks:

- `base_low` (Blocks 1–2): learns edges and textures. Frozen permanently after Task 0.
- `base_high` (Block 3 + FC): learns higher-level structure. Also frozen after Task 0 for CIFAR; kept adaptive at $0.1\times$ learning rate for Permuted MNIST.

Freezing the base encoder is the key design decision: subsequent tasks *cannot* modify the frozen shared representation, which prevents direct representational overwriting in the evaluated task-incremental setting.

3.2.2 Task-Specific Adapter Modules

For each task t, a lightweight adapter `TaskModule[`t`]` processes the raw input independently:

$$
\begin{aligned}
h_1 &= \mathrm{MaxPool}\big(\mathrm{ReLU}(\mathrm{BN}(\mathrm{Conv}_{3\to 32}(x)))\big) \\
h_2 &= \mathrm{MaxPool}\big(\mathrm{ReLU}(\mathrm{BN}(\mathrm{Conv}_{32\to 64}(h_1)))\big) \\
h_3 &= \mathrm{AvgPool}\big(\mathrm{ReLU}(\mathrm{BN}(\mathrm{Conv}_{64\to 64}(h_2)))\big) \\
\mathbf{s} &= \sigma(g) \cdot W_s \,\mathrm{flatten}(h_3)
\end{aligned}
$$

A scalar gate g is initialised to -3.0 so that $\sigma(g) \approx 0.05$; the module starts with near-zero contribution and opens as gradients prove it useful.

3.2.3 Attention Router

The router fuses $\mathbf{b} \in \mathbb{R}^{512}$ (base) and $\mathbf{s} \in \mathbb{R}^{256}$ (task) via learned per-sample attention:

$$
\begin{aligned}
\mathbf{w} &= \mathrm{Softmax}\big(\mathrm{MLP}([\mathbf{b} \| \mathbf{s}])\big) \in \mathbb{R}^2 \\
\mathbf{f} &= w_0 \cdot W_b\,\mathbf{b} \;+\; w_1 \cdot \mathbf{s} \\
\hat{\mathbf{f}} &= \mathrm{LayerNorm}(W_f\,\mathbf{f} + \mathbf{f})
\end{aligned}
$$

This allows the network to upweight the base stream when general features suffice and the task stream when specialised patterns are needed—on a per-sample basis.

3.2.4 Per-Task Heads

Each task t has an independent linear classifier $\mathrm{Head}[t]: \mathbb{R}^{256} \to \mathbb{R}^{C_t}$. Independent heads ensure zero cross-task interference at the classification layer.

3.3 Training Procedure

Algorithm 1 summarises the training loop. Because frozen parameters receive no gradients, and all unfrozen parameters are exclusive to the current task, training on task t does not directly update the components used by tasks $0, \ldots, t-1$ in the task-incremental setting.

Algorithm 1 MCN Training

for task $t = 0, 1, \ldots, T-1$ **do**
  Initialise `TaskModule[`t`]`, `Router[`t`]`, `Head[`t`]`
  Freeze all modules for tasks $0, \ldots, t-1$
  Freeze `base_low`; freeze or reduce lr of `base_high`
  for epoch $= 1, \ldots, E$ **do**
    for each mini-batch $(x, y) \sim \mathcal{D}_t$ **do**
      $\hat{y} = \mathrm{MCN}(x;\ t)$
      $\mathcal{L} = \mathrm{CrossEntropy}(\hat{y},\ y)$
      Update *only* unfrozen parameters
    end for
  end for
end for

4 Experiments

4.1 Benchmarks

Split-CIFAR-10 (5 tasks). CIFAR-10 split into five binary tasks: {airplane, automobile}, {bird, cat}, {deer, dog}, {frog, horse}, {ship, truck}. 10 k train / 2 k test images per task at 32×32.

Split-CIFAR-100 (20 tasks). CIFAR-100 split into twenty 5-way tasks. 2.5 k train / 500 test per task. The hardest standard benchmark: $4\times$ more tasks, finer-grained classes, less data per task.

Permuted MNIST (5 tasks). MNIST with a fixed random pixel permutation per task. All methods use an MLP backbone since spatial structure is destroyed.

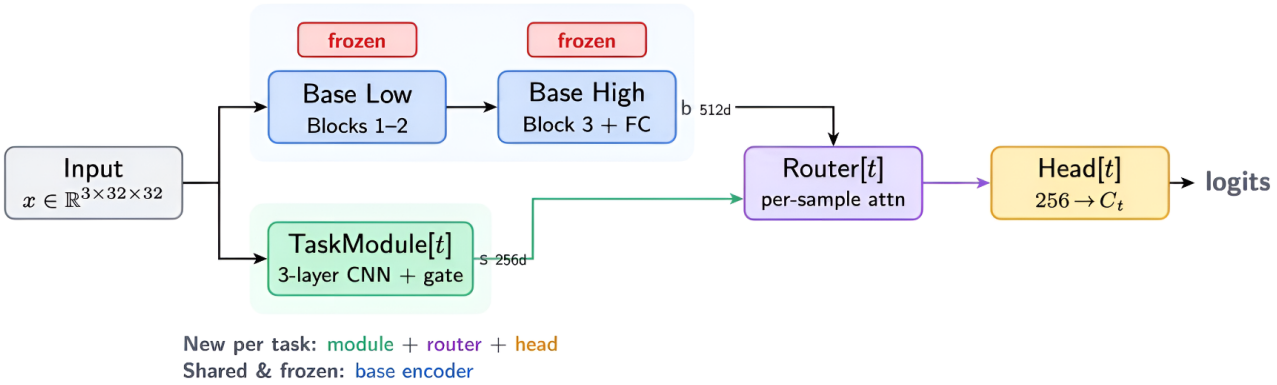

Figure 1: **MCN architecture overview.** The input is processed in parallel by a frozen shared base encoder (trained on Task 0, then permanently frozen) and a task-specific adapter module (newly initialised per task). A learned attention router computes per-sample weights $w_{\text{base}}, w_{\text{task}}$ to blend the two feature streams. A task-specific head produces the final logits. All previously learned modules are frozen during new-task training; in the evaluated task-incremental setting, this prevents direct gradient interference with older task components.

Table 1: Split-CIFAR-10 (5 tasks).

Method	Avg Acc↑	BWT↑	Forgetting↓
Naive	50.7%	−55.6%	55.6%
HAT	59.4%	−43.7%	43.7%
EWC	60.8%	−39.2%	39.2%
PackNet	77.0%	−9.3%	9.3%
MCN	**92.7%**	**−0.1%**	**0.1%**

Table 2: Split-CIFAR-100 (20 tasks).

Method	Avg Acc↑	BWT↑	Forgetting↓
Naive	23.8%	−64.4%	64.4%
PackNet	56.2%	−3.3%	4.5%
EWC	66.0%	−11.2%	11.3%
MCN	**75.1%**	**−1.1%**	**1.5%**

4.2 Baselines

Naive: sequential SGD, no forgetting mitigation (lower bound). **EWC** [2]: $\lambda = 5000$, Fisher estimated on the full training set. **PackNet** [3]: two-phase train/retrain, prune fraction = 0.5. **HAT** [5]: learned attention masks, $s_{\max} = 400$ with annealing.

4.3 Metrics

Average Accuracy (AA): $\frac{1}{T}\sum_t R_{T-1,t}$ (higher = better). **Backward Transfer (BWT)**: mean accuracy drop from training time to final evaluation (less negative = better). **Forgetting (FM)**: average drop from peak per-task accuracy (lower = better).

Table 3: Permuted MNIST (5 tasks). EWC is competitive on structurally homogeneous tasks.

Method	Avg Acc↑	BWT↑	Forgetting↓
Naive	87.5%	−12.8%	12.8%
PackNet	95.4%	−3.0%	3.0%
MCN	95.9%	−1.2%	1.2%
EWC	**96.8%**	**−1.0%**	**1.0%**

4.4 Main Results

On **Split-CIFAR-10** (Table 1), MCN surpasses all baselines by a wide margin. Task 0 accuracy at the end of training is within 0.1 pp of its accuracy immediately after Task 0 training—near-perfect retention. HAT, despite learned attention masking, achieves only 59.4% with 43.7% forgetting, worse than EWC on this benchmark.

On **Split-CIFAR-100** (Table 2), the 20-task setting amplifies the differences. Naive collapses to 23.8%. PackNet preserves old tasks well (4.5% forgetting) but lacks capacity for new ones (56.2% accuracy). EWC holds at 66.0% but accumulates 11.3% forgetting over 19 prior Fisher matrices. MCN achieves 75.1% accuracy with just 1.5% forgetting in this task-incremental evaluation—showing that the modular design scales well to long task sequences.

On **Permuted MNIST** (Table 3), EWC wins. This is an honest finding: when all tasks share the same low-level structure, soft regularisation suffices. MCN's modular overhead adds complexity without proportional benefit in this regime.

4.5 Forgetting Dynamics

Figure 2 visualises the forgetting pattern across methods on Split-CIFAR-10.

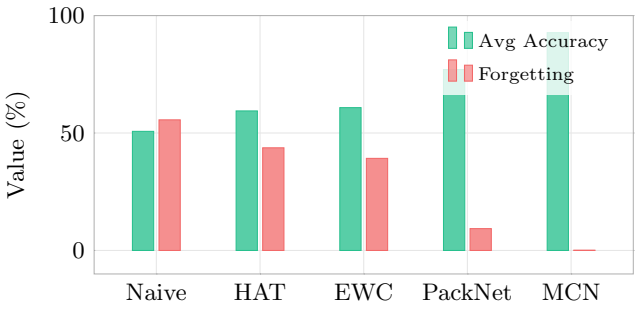

Figure 2: **Split-CIFAR-10 summary.** MCN achieves the highest accuracy with near-zero forgetting. Methods ordered left-to-right by average accuracy.

4.6 Accuracy Matrices

Table 4 presents accuracy matrices on Split-CIFAR-10. Entry $R_{i,j}$ records test accuracy on task j after training through task i.

Table 4: Accuracy matrices on Split-CIFAR-10 (5 tasks). MCN retains near-constant accuracy for all past tasks; HAT degrades severely.

MCN

	T0	T1	T2	T3	T4
$t=0$	**97**	–	–	–	–
$t=1$	97	85	–	–	–
$t=2$	97	85	90	–	–
$t=3$	97	85	90	97	–
$t=4$	97	84	90	97	95

HAT

	T0	T1	T2	T3	T4
$t=0$	97	–	–	–	–
$t=1$	50	90	–	–	–
$t=2$	66	54	93	–	–
$t=3$	52	51	65	97	–
$t=4$	**32**	50	57	63	95

Naive

	T0	T1	T2	T3	T4
$t=0$	97	–	–	–	–
$t=1$	67	91	–	–	–
$t=2$	71	69	94	–	–
$t=3$	51	54	58	98	–
$t=4$	**24**	40	47	46	97

MCN's matrix is uniformly high: Task 0 stays at 97% throughout. Naive shows the classic collapse—Task 0 drops from 97% to 24% (near chance for binary classification). HAT follows a similar trajectory, reaching 32% on Task 0 despite its attention masks.

5 Ablation Study

We run ablations on Split-CIFAR-10 with 3 tasks (Table 5).

Table 5: Ablation on 3-task Split-CIFAR-10.

Variant	Acc	Forg.	Takeaway
MCN (full)	**89.2%**	0.3%	Full architecture
– Router	87.7%	0.1%	Router: +1.5% acc
– Gate	89.7%	0.0%	Gate stabilises training
– Modules	71.7%	0.4%	Modules: +17.5% acc

Task modules are essential. Removing them drops accuracy by 17.5 pp. The frozen base alone cannot capture task-specific patterns; each adapter learns complementary representations.

The router improves accuracy. Replacing the learned attention with fixed-weight concatenation costs 1.5 pp with no forgetting benefit. Per-sample weighting is valuable—different inputs benefit from different base-vs-task ratios.

The gate stabilises training. MCN-NoGate is comparable in final accuracy but the near-zero initialisation ($\sigma(-3) \approx 0.05$) prevents early-training oscillation when random task-module weights could disrupt the base features' gradient signal.

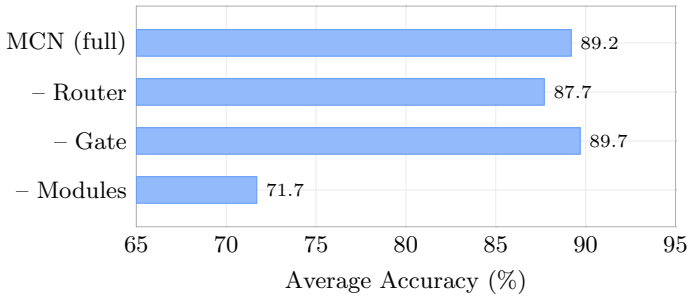

Figure 3: **Ablation results.** Removing task modules causes the largest accuracy drop (−17.5 pp), confirming they are the most critical component.

6 Analysis

6.1 Why MCN Outperforms Regularisation Methods

EWC's total loss after T tasks contains a sum of Fisher penalties from all previous tasks:

$$\mathcal{L} = \mathcal{L}_T + \frac{\lambda}{2} \sum_{t<T} \sum_i F_{t,i} \, (\theta_i - \theta^*_{t,i})^2$$

These constraints conflict: a parameter important for task 0 may need to change for task T. EWC's 39.2% forgetting on CIFAR-10 and 11.3% on CIFAR-100 reflect

this compounding. MCN avoids this form of direct interference by design: frozen parameters receive no gradients, and architectural separation is not overridden by gradient magnitude.

6.2 Why MCN Outperforms Masking Methods

PackNet's capacity is finite: with 50% prune rate, free capacity halves per task ($50\% \to 25\% \to 12.5\% \to 6.25\%$). This is visible in the CIFAR-100 results where PackNet falls to 56.2%. HAT's softer masks share capacity but still show 43.7% forgetting on CIFAR-10 when tasks are visually distinct. MCN faces no fixed-capacity ceiling: each new $\sim$1.47 M-parameter module provides fresh capacity, at the cost of linear model growth with the number of tasks.

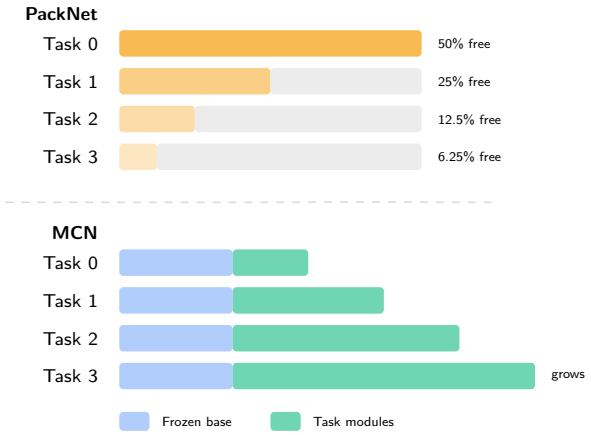

Figure 4: **Capacity comparison.** PackNet's available capacity halves per task within a fixed budget. MCN grows linearly—each task adds fresh capacity without depleting a shared pool.

6.3 Scalability: 5 vs. 20 Tasks

MCN's forgetting rises only from 0.1% (5 tasks) to 1.5% (20 tasks). The modest increase is attributable to the base encoder being trained on only the first of 20 tasks—a narrower initial training signal.

6.4 Parameter Efficiency

Table 6: Parameter budget for MCN on CIFAR (32×32).

Component	Parameters
Base encoder (shared, frozen)	≈ 3.24 M
Per-task module + router	≈ 1.47 M
Per-task head	≈ 2 K
Total (5 tasks)	≈ 10.60 M
Total (20 tasks)	≈ 32.68 M

Growth is linear and bounded per task (Table 6). While larger than fixed-capacity methods, the accuracy gains justify the cost in these experiments, and module pruning/merging (future work) can reduce it further.

6.5 Limitations

1. **Task identity at inference.** The headline evaluation is task-incremental: MCN requires knowing which task is active at inference time. Class-incremental learning would need an additional task-inference mechanism.
2. **Linear parameter growth.** For very long sequences ($T > 100$), storage may become a concern without module compression.
3. **Homogeneous-task parity.** MCN does not outperform EWC when tasks share the same low-level structure (Permuted MNIST).
4. **Base encoder quality.** Performance depends on Task 0 training quality. An unrepresentative first task yields suboptimal frozen features for later tasks.
5. **Single-run results.** Reported results are deterministic single-run experiments; multi-seed mean and standard deviation evaluation is future work.
6. **Baseline scope.** The baseline implementations are educational/research prototypes, so absolute numbers may vary with further tuning or alternative implementations.

7 Conclusion

We introduced the Modular Continual Network (MCN), a task-incremental continual learning architecture that achieves near-zero catastrophic forgetting in these experiments by growing modular capacity per task. The core principle—freeze a shared encoder after Task 0 and grow task-specific adapters for each new task—removes direct gradient interference with previously learned task components in the evaluated task-incremental setting.

MCN achieves 92.7% accuracy with 0.1% forgetting on Split-CIFAR-10 and scales to 75.1% accuracy with 1.5% forgetting on the 20-task Split-CIFAR-100 benchmark, surpassing EWC, PackNet, and HAT in these experiments. The result suggests a broader design principle: rather than only protecting shared weights from damage, architect systems where new tasks minimise interference with old representations. Structural separation can be more reliable than penalty-based discouragement in task-incremental settings.

Future work. (1) Cross-task knowledge sharing via inter-module attention (MCN v2). (2) Task-free inference via learned task identification. (3) Module pruning and merging for long-horizon efficiency. (4) Application to transformer architectures for language continual learning.

Reproducibility

All experiments were conducted on an Apple M4 MacBook with MPS acceleration and deterministic random seeds. The reported results are single-run measurements; multi-seed mean and standard deviation evaluation is future work. Full code, trained models, and evaluation scripts are available at https://github.com/sire-magnusss/modular-continual-network.

Table 7: Hyperparameters.

Parameter	Value	Note
Learning rate	10^{-3}	Adam, all methods
Base_high lr (MCN)	10^{-4}	0.1×, MNIST only
Batch size	128	
Epochs / task	10	
EWC λ	5 000	
PackNet prune rate	0.50	per task
HAT $s_{\max}$	400	sparsity coeff.
Gate initialisation	-3.0	$\sigma(-3) \approx 0.05$
Base encoder dim	512	
Task module dim	256	
Router hidden dim	128	

References

[1] McCloskey, M. & Cohen, N. J. (1989). Catastrophic interference in connectionist networks: The sequential learning problem. *Psych. of Learning and Motivation*, 24, 109–165.

[2] Kirkpatrick, J., Pascanu, R., Rabinowitz, N. et al. (2017). Overcoming catastrophic forgetting in neural networks. *PNAS*, 114(13), 3521–3526.

[3] Mallya, A. & Lazebnik, S. (2018). PackNet: Adding multiple tasks to a single network by iterative pruning. *CVPR*.

[4] Rusu, A. A., Rabinowitz, N. C., Desjardins, G. et al. (2016). Progressive neural networks. *arXiv:1606.04671*.

[5] Serra, J., Suris, D., Miron, M. & Karatzoglou, A. (2018). Overcoming catastrophic forgetting with hard attention to the task. *ICML*.

[6] Yoon, J., Yang, E., Lee, J. & Hwang, S. J. (2018). Lifelong learning with dynamically expandable networks. *ICLR*.